# Deployment and Operations Runbook

## Swasthya Katha — Safe Release and Reliable Operations

**Version:** 1.0
**Status:** Draft for DevOps and operations approval
**Date:** 24 August 2026

### 1. Purpose

This runbook defines how code, content, assets, translations, and configuration move from development to production and how the team responds to failures.

### 2. Environment model

```
Local
Development
Staging
Clinical review
Pilot
Production
```

Use separate credentials and storage policies for each environment.

### 3. Branch model

```
main        production-ready
staging     release candidate
feature/*   application work
content/*   content changes
hotfix/*    urgent corrections
```

### 4. Pull request requirements

Every pull request must include:

- Related ticket.
- User outcome.
- Summary of change.
- Tests run.
- Screenshots or recording for visual changes.
- Accessibility impact.
- Security impact.

- Performance impact.
- Clinical/content impact.
- Rollback plan.

### 5. CI pipeline

```
Install dependencies
  -> Type check
  -> Lint
  -> Unit tests
  -> Content validation
  -> Security checks
  -> Build
  -> E2E smoke tests
  -> Accessibility smoke tests
  -> Preview deployment
```

The pipeline must fail if:

- Required schema is invalid.
- Secrets are detected.
- Critical test fails.
- Unapproved content is in the publication manifest.
- Unsafe asset is detected.

### 6. Environment configuration

Validate environment variables at startup:

```
NEXT_PUBLIC_APP_URL
NEXT_PUBLIC_DEFAULT_LOCALE
NEXT_PUBLIC_SUPABASE_URL
NEXT_PUBLIC_SUPABASE_ANON_KEY
SUPABASE_SERVICE_ROLE_KEY
PUBLIC_ASSET_BUCKET
DRAFT_ASSET_BUCKET
AUDIO_BUCKET
ERROR_MONITORING_DSN
```

Never commit secrets or use production credentials locally.

### 7. Staging verification

Verify:

- Public routes.
- Admin routes.

- All launch locales.
- Published-only filtering.
- Magazine and linear modes.
- Keyboard navigation.
- Audio and transcript.
- SVG scenes.
- WebGL fallback.
- Offline lesson.
- Error states.
- Privacy controls.
- Analytics failure isolation.

### 8. Release package

Every release package includes:

```
Application commit:
Publication version:
Locale list:
Database migration version:
Asset manifest:
Clinical approval:
Language approval:
Accessibility approval:
Security approval:
Performance approval:
Rollback target:
Known limitations:
Release owner:
```

### 9. Production release sequence

1. Freeze the release candidate.
2. Confirm release scorecard.
3. Confirm content version and locale list.
4. Confirm rollback target.
5. Run final CI.
6. Deploy application.
7. Apply reviewed migrations.
8. Publish immutable content manifest.
9. Run smoke tests.
10. Monitor errors and asset failures.

11. Record release in project memory.

## 10. Rollback sequence

1. Identify application, content, or asset failure.
2. Disable affected feature or publication if necessary.
3. Revert to last approved application version.
4. Revert publication manifest or individual asset.
5. Run public-reader smoke test.
6. Notify product and clinical owners for content issues.
7. Create corrective ticket.
8. Update release log and project memory.

Rollback must be tested before launch, not invented during an incident.

## 11. Monitoring

Monitor:

- Error rate.
- Failed publication loads.
- Asset failures.
- Audio failures.
- Visual fallback rate.
- WebGL initialization failures.
- Offline cache errors.
- Build and deployment failures.
- Unauthorized access attempts.
- Missing translation errors.

Avoid tracking sensitive reading topics against individual identities.

## 12. Incident response

### 12.1 Safety/content incident

- Disable affected content.
- Notify clinical owner.
- Preserve version.
- Assess impact.
- Correct content.
- Re-review.

- Publish corrected release.
- Document root cause.

### 12.2 Security/privacy incident

- Contain exposure.
- Revoke credentials or URLs.
- Preserve evidence.
- Identify affected data.
- Follow applicable notification obligations.
- Patch and test.
- Record corrective actions.

### 12.3 Availability incident

- Confirm scope.
- Check deployment and storage.
- Enable cached/static fallback where possible.
- Roll back or restore.
- Verify core reader.
- Monitor recovery.

## 13. Scheduled operations

### Daily

- Review error alerts.
- Review failed asset loads.
- Check active incidents.
- Update project memory for active work.

### Weekly

- Review release candidates.
- Review open P0/P1 tickets.
- Review content and translation status.
- Check backup health.

### Monthly

- Review access roles.
- Review dependencies.
- Review analytics and data retention.
- Review performance trends.

### On content review date

- Identify due lessons.
- Assign reviewers.
- Block outdated release if required.
- Publish updated version only after approval.

## 14. Operational acceptance criteria

- Deployment is repeatable.
- Rollback is documented and tested.
- Monitoring is active.
- Staff access is protected.
- Publication version is visible.
- Current lesson fallback works.
- Incident owners are named.
- Release log is complete.